UPDATE: modificar registros y la regla de oro del WHERE

Los datos cambian. Un cliente actualiza su correo, un producto sube de precio, un pedido cambia de estado. `UPDATE` es el comando que modifica registros que ya existen en una tabla, sin necesidad de borrarlos y volver a insertarlos.

Pero antes de ver cómo funciona, hay una advertencia que vale repetir desde el principio: **un `UPDATE` sin `WHERE` modifica todas las filas de la tabla, sin excepción, y no hay forma de deshacer ese cambio**. No existe un Ctrl+Z en SQL. Por eso esta clase empieza y termina con la misma regla.

La regla de oro: UPDATE siempre con WHERE

```sql
-- ✅ Correcto: modifica solo la fila indicada
UPDATE clientes
SET activo = FALSE
WHERE cliente_id = 1;


-- ❌ Peligroso: desactiva TODOS los clientes de la tabla
UPDATE clientes
SET activo = FALSE;
```

Ambas instrucciones son sintácticamente válidas. El motor ejecuta las dos sin error. La diferencia es que la segunda afecta a todos los registros de la tabla, no solo al que querías cambiar. En una base de datos con miles de clientes, ese error puede ser catastrófico.

La regla es simple: antes de ejecutar cualquier `UPDATE`, verifica que el `WHERE` esté ahí y que apunte exactamente a lo que quieres modificar.

UPDATE básico: modificar una columna

La estructura del comando es: `UPDATE`, nombre de la tabla, `SET`, el campo que cambia con su nuevo valor, y `WHERE`, la condición que identifica qué filas se modifican.

```sql
UPDATE clientes

SET activo = FALSE

WHERE cliente_id = 1;
```

Esto cambia el campo `activo` a `FALSE` únicamente en el cliente con ID igual a 1. El resto de los clientes no se toca.

Para confirmar que el cambio se aplicó, puedes hacer un `SELECT` después:

```sql
SELECT cliente_id, nombre, activo

FROM clientes

WHERE cliente_id = 1;
```

UPDATE de múltiples columnas

Si necesitas modificar más de un campo en la misma operación, los separas con coma dentro del `SET`:

```sql
UPDATE clientes

SET nombre = 'María González Reyes',

    activo = TRUE
```

```
WHERE cliente_id = 1;
```

Con una sola instrucción actualizaste el nombre y la condición de activo al mismo tiempo, en la misma fila. El `WHERE` sigue siendo obligatorio y aplica a todos los campos del `SET`.

UPDATE con RETURNING: verificar el cambio en el momento

Ya usaste `RETURNING` con `INSERT`. Con `UPDATE` funciona igual: le pides al motor que te devuelva los datos de las filas que acaba de modificar, justo después de hacer el cambio.

```SQL
UPDATE productos

SET precio = precio * 1.10

WHERE categoria_id = 1

RETURNING producto_id, nombre, precio;
```

Esta instrucción aumenta el precio de todos los productos de la categoría 1 en un 10% y devuelve el ID, nombre y precio nuevo de cada uno. Sin necesidad de hacer un `SELECT` por separado.

`RETURNING` es especialmente útil cuando el nuevo valor se calcula a partir del valor anterior, como en este caso, porque te confirma exactamente qué resultado produjo la operación.

UPDATE con condiciones más específicas

El `WHERE` del `UPDATE` acepta las mismas condiciones que ya viste en `SELECT`: comparadores, rangos, listas, `AND`, `OR`. Eso te permite ser muy preciso sobre qué filas modificar.

Actualizar por rango:

```sql
UPDATE productos

SET precio = precio * 0.90

WHERE precio > 500 AND stock > 0;
```

Aplica un descuento del 10% solo a los productos con precio mayor a 500 y que tengan stock disponible.

Actualizar con lista:

```sql
UPDATE clientes

SET tipo_cliente_id = 2

WHERE cliente_id IN (3, 5, 8);
```

Cambia el tipo de cliente únicamente para los tres IDs indicados.

El hábito correcto antes de ejecutar un UPDATE

Antes de correr cualquier `UPDATE`, hay un paso que conviene convertir en rutina: primero ejecuta un `SELECT` con el mismo `WHERE` para ver exactamente qué filas vas a modificar.

```sql
-- Primero verificas qué filas van a cambiar:

SELECT cliente_id, nombre, activo

FROM clientes

WHERE cliente_id = 1;
```

```
-- Si el resultado es el correcto, ejecutas el UPDATE:
UPDATE clientes
SET activo = FALSE
WHERE cliente_id = 1;
```

Este hábito toma dos segundos y puede evitar un error que tome horas corregir. Si el `SELECT` devuelve las filas correctas, el `UPDATE` con el mismo `WHERE` va a modificar exactamente esas mismas filas, ni una más.

Resumen del comando

Qué quieres hacer	Cómo escribirlo
Modificar un campo	`SET campo = valor WHERE condicion`
Modificar varios campos a la vez	`SET campo1 = valor1, campo2 = valor2 WHERE ...`
Calcular el nuevo valor desde el actual	`SET precio = precio * 1.10 WHERE ...`
Verificar el resultado sin SELECT aparte	Agregar `RETURNING columnas` al final

Modificar datos es una operación que el motor ejecuta de inmediato y sin confirmación. La siguiente clase cubre `DELETE`, el comando para eliminar filas, donde esa misma lógica aplica con consecuencias todavía más permanentes.